
Distance Sensing and Occupancy Grid Mapping

AI and Robotics

Building environmental maps from LiDAR sensor data



Today's Agenda

01

Distance Sensing Technologies

Compare LiDAR, ultrasonic, and infrared sensors for robot perception

02

LiDAR and Point Clouds

Understand how LiDAR works and represents environmental data

03

Occupancy Grid Representation

Learn probabilistic grid maps and log-odds representation

04

Mapping Algorithm

Implement inverse sensor model and ray casting for grid updates

05

Real-World Applications

Explore applications in warehouses, autonomous vehicles, and beyond

Why Mapping Matters for Autonomous Robots

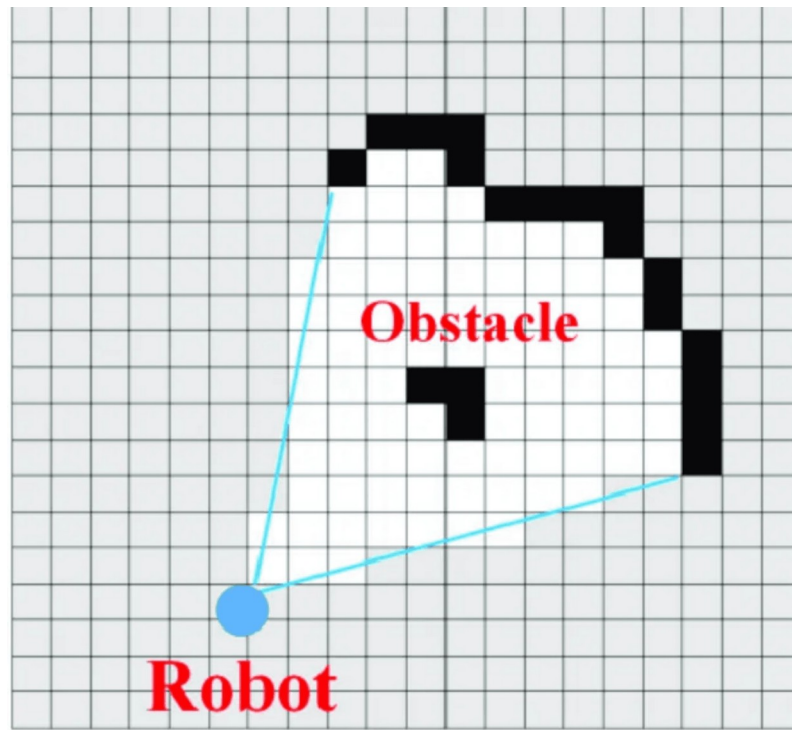
Week 3: We learned how robots answer *"Where am I?"* using Kalman filtering for localization.

Week 4: Now we tackle *"What's around me?"* by building maps from sensor data.

Localization + Mapping = Autonomous Navigation

Real-World Example: A warehouse robot must know obstacle locations to plan safe paths for package delivery.

The Challenge: Build accurate, reliable maps from noisy sensor measurements in real-time.



Example occupancy grid map showing obstacles (black), free space (white), and unknown areas (gray).

Distance Sensing Technologies: Three Main Types

Robots use different sensors depending on application requirements

Characteristic	Ultrasonic	Infrared (IR)	LiDAR
Technology	Sound waves (40 kHz)	Infrared light reflection	Laser pulses (TOF)
Range	2 cm – 4 m	10 cm – 80 cm	0.1 m – 100+ m
Accuracy	±1-3 cm	±2-5 cm	±2-5 cm (long range) ±1 mm (short range)
Update Rate	≤50 Hz	100+ Hz	5-40 Hz (360° scan)
Cost	\$1-5	\$2-10	\$100-\$10,000+
Key Advantage	Cheap, simple, robust	Fast, compact	Accurate, 360° coverage
Key Limitation	Slow, wide beam angle	Lighting sensitive, short range	Expensive, complex

Ultrasonic Sensors: Low-Cost Distance Measurement

TECHNOLOGY

Sound waves at 40 kHz (above human hearing)

RANGE

2 cm to 4 meters (typical)

ACCURACY

$\pm 1-3$ cm

UPDATE RATE

Up to 50 Hz (20 ms per measurement)

COST

\$1-5 per sensor

EXAMPLE

HC-SR04 (Arduino), MaxBotix sensors

Advantages

- Very low cost and simple to use
- Works in various lighting conditions (day/night)
- Not affected by object color or transparency
- Robust to dust and smoke

Disadvantages

- Slow update rate (limited by speed of sound)
- Wide beam angle ($\sim 15^\circ$) reduces precision
- Affected by soft materials (curtains, foam)
- Temperature dependent (speed of sound varies)

Applications

- Car parking sensors
- Low-cost mobile robots
- Obstacle detection for Arduino projects

Infrared Sensors: Fast but Limited Range

TECHNOLOGY

Infrared light reflection (IR LED + photodiode)

RANGE

10 cm to 80 cm (typical)

ACCURACY

±2-5 cm

UPDATE RATE

100+ Hz (very fast)

COST

\$2-10 per sensor

EXAMPLE

Sharp GP2Y0A21YK, Pololu distance sensors

Advantages

- Very fast update rate (100+ Hz)
- Low cost and compact size
- Simple analog output (voltage proportional to distance)
- Good for close-range detection

Disadvantages

- Affected by ambient light (sunlight, IR sources)
- Surface color and texture affect readings
- Limited range (typically <1 meter)
- Nonlinear response curve (requires calibration)

Applications

- Line-following robots
- Cliff detection (vacuum cleaners)
- Proximity sensing for user interfaces

LiDAR: Industry Standard for Robot Mapping

TECHNOLOGY

Laser pulses (infrared or visible light)

RANGE

0.1 m to 100+ meters (model dependent)

ACCURACY

±2-5 cm at long range, ±1 mm at short range

UPDATE RATE

5-40 Hz for full 360° scan

RESOLUTION

360-1080 measurements per scan

COST

\$100 (RPLiDAR) to \$10,000+ (Velodyne)

EXAMPLES

SICK LMS, Hokuyo URG, RPLiDAR, Velodyne

Advantages

- High accuracy and long range
- 360° coverage in single scan
- Not affected by lighting conditions
- Dense point clouds for detailed mapping

Disadvantages

- Expensive (especially high-end models)
- Complex data processing required
- Struggles with transparent surfaces (glass, water)
- Reflective surfaces cause false readings

Applications

- Autonomous vehicles (Tesla, Waymo)
- Warehouse robots (Amazon, Fetch)

Drones and UAVs

How LiDAR Measures Distance

1

Emit a laser pulse toward the environment

2

Measure time-of-flight (TOF): time for light to travel to object and return

3

Calculate distance using the speed of light

$$d = (c \times t) / 2$$

where c = speed of light (3×10^8 m/s), t = time-of-flight

4

Rotate sensor to scan 360° around the robot

5

Result: Array of 360-1080 distance measurements at different angles

Point Cloud Representation of Environment

LiDAR Output: Array of (angle, distance) pairs for each measurement in a scan.

Conversion to Cartesian: Transform polar coordinates to (x, y) in robot frame:

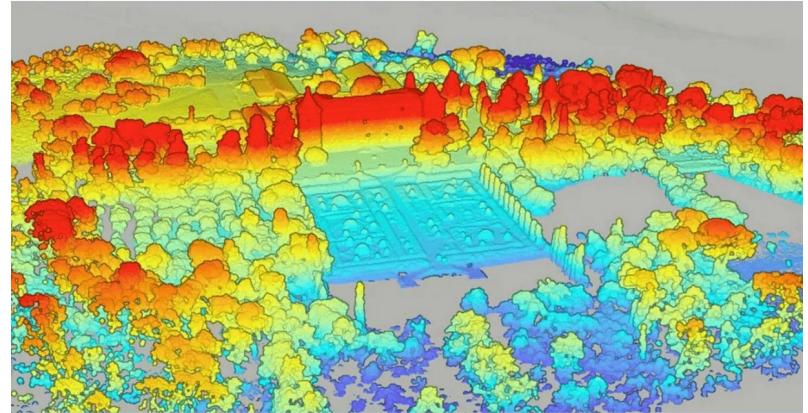
$$x = d \times \cos(\theta)$$

$$y = d \times \sin(\theta)$$

Point Cloud: Set of (x, y) points representing detected obstacles in the environment.

Visualization: Scatter plot where each point represents a surface the laser hit.

Challenge: How do we use this sparse point data for



3D point cloud from LiDAR showing environmental structure

Coordinate Frame Transformations

Robot Frame

Origin at robot center

x-axis: forward

y-axis: left

LiDAR measurements in this frame

World Frame

Fixed global coordinate system

Independent of robot motion

Map is built in this frame

Persistent across time

Transformation: Robot $\rightarrow$ World

Given robot pose (x^r, y^r, θ^r) in world frame:

$$x_{\text{world}} = x_r + x_{\text{robot}} \times \cos(\theta_r) - y_{\text{robot}} \times \sin(\theta_r)$$

$$y_{\text{world}} = y_r + x_{\text{robot}} \times \sin(\theta_r) + y_{\text{robot}} \times \cos(\theta_r)$$

Occupancy Grid Maps: Discretizing the Environment

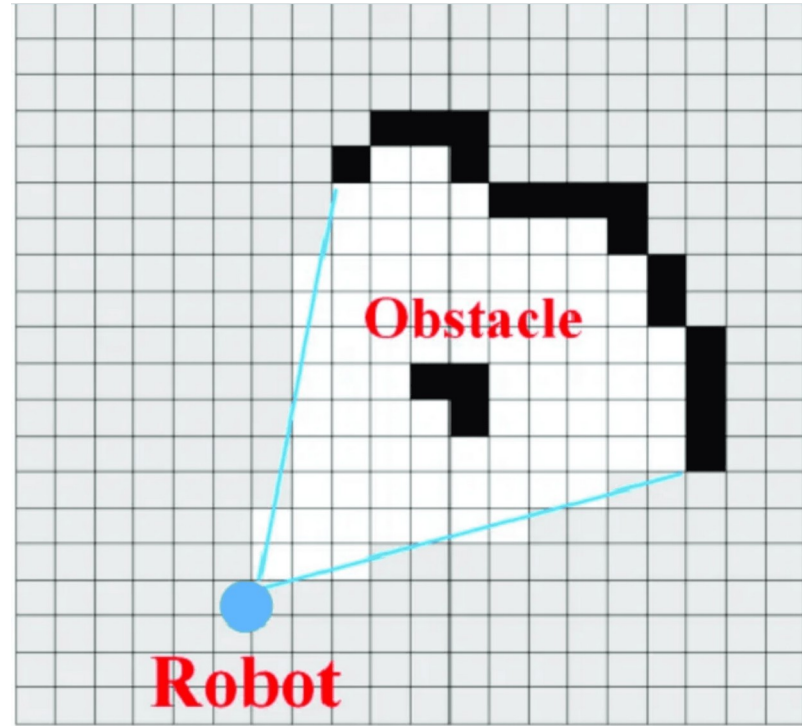
Core Idea: Discretize the continuous environment into a grid of cells (e.g., 10 cm × 10 cm).

Each Cell Stores: Probability that the cell is occupied:
 $P(\text{occupied})$

Three States:

Occupied	$P > 0.7$ (obstacle present)
Free	$P < 0.3$ (no obstacle)
Unknown	$0.3 \leq P \leq 0.7$ (uncertain)

Advantages: Simple, efficient, handles uncertainty naturally



Grid Data Structure and Parameters

Implementation: 2D array where `grid[x][y]` stores the probability of cell (x, y) being occupied.

Resolution

Size of each cell (e.g., 0.1 m = 10 cm)

Trade-off: Higher resolution = more detail but more memory and computation

Size

Width × Height in cells

Example: 100 × 100 cells = 10m × 10m area

Depends on environment size

Origin

Where is `grid[0][0]` in world coordinates?

Typically center or corner of the mapped environment

Data Type

Float (4 bytes) for probability
OR log-odds representation

Can use int8 for memory efficiency

Grid Memory Requirements and Visualization

Memory Example:

100×100 grid $\times$ 1 byte per cell = **10,000 bytes = 10 KB**

1000×1000 grid $\times$ 1 byte per cell = 1,000,000 bytes = 1 MB

Even large environments remain memory-efficient!

Visualization as Grayscale Image

Black (P = 1.0): Occupied cell (obstacle present)

White (P = 0.0): Free cell (no obstacle)

Gray (P = 0.5): Unknown cell (not yet explored)

Log-Odds: Efficient Probability Updates

Problem with Probabilities

Updating probabilities requires multiplication, which is slow and prone to numerical underflow when values approach 0 or 1.

Solution: Log-Odds Representation

$$L(x) = \log(P(x) / (1 - P(x)))$$

Convert probability to log-odds, update using addition, convert back to probability when needed.

Advantages:



Addition instead of multiplication: $L_{\text{new}} = L_{\text{old}} + L_{\text{measurement}}$



Unbounded range: $(-\infty, +\infty)$ instead of $[0, 1]$

Inverse Sensor Model for LiDAR

Concept: Given a measurement, what is the probability that a cell is occupied? $P(\text{occupied} \mid \text{measurement})$

For each LiDAR ray: Update all cells along the ray from robot to measured endpoint.

Update Rules:

1. Endpoint (Obstacle Hit)

Cell where laser hit an obstacle → **Increase occupancy** (e.g., $P = 0.7$ or $L = +1.0$)

2. Before Endpoint (Free Space)

Cells between robot and endpoint → **Decrease occupancy** (e.g., $P = 0.3$ or $L = -1.0$)

Laser passed through, so these cells are free

3. After Endpoint (Unknown)

Cells beyond endpoint → **No update**

Laser didn't reach these cells, so we have no information

Ray Casting: Finding Cells Along a Ray

Problem: Given start (robot) and end (measurement) points, which grid cells does the ray pass through?

Bresenham's Line Algorithm

Classic algorithm from computer graphics for drawing lines on pixel grids

Input: Start (x_0, y_0) , End (x_1, y_1)

Output: List of all grid cells along the line

Efficiency

Uses only integer arithmetic (addition, subtraction)

No floating-point operations or trigonometry

Very fast, suitable for real-time mapping

Usage in Mapping

1. Ray cast from robot to measurement endpoint
2. Mark all cells along ray as "free"
3. Mark endpoint cell as "occupied"
4. Repeat for all measurements in scan

Implementation: Available in most robotics libraries (e.g., Python's scikit-image, ROS nav_msgs)

Occupancy Grid Update Algorithm

1

Get measurement: (angle, distance) from LiDAR scan

2

Convert to Cartesian: $(x, y) = (d \times \cos(\theta), d \times \sin(\theta))$ in robot frame

3

Transform to world frame: Apply robot pose (x_r, y_r, θ_r) transformation

4

Ray cast: Find all grid cells from robot position to measurement endpoint

5

Update cells: For each cell along ray:

- Before endpoint: $L_{\text{cell}} += L_{\text{free}}$ (e.g., -1.0)
- At endpoint: $L_{\text{cell}} += L_{\text{occupied}}$ (e.g., $+1.0$)

6

Clamp values: Limit log-odds to prevent overflow (e.g., $[-10, +10]$)

7

Repeat: Process all measurements in scan, then all scans over time

Result: Occupancy grid map showing obstacles, free space, and unknown areas

Mapping with Known Robot Poses

Key Assumption: We assume the robot's pose (x, y, θ) is known at each time step from localization (Week 3: Kalman filtering).

Input

- Sequence of robot poses
- LiDAR scans at each pose
- Sensor parameters (noise, range)

Output

- Occupancy grid map
 - Black: obstacles
 - White: free space
- Gray: unknown

Process

1. Initialize empty grid (all unknown)
2. For each (pose, scan) pair:
 - Transform measurements to world frame
 - Update grid using inverse sensor model
3. Result: Map showing obstacles and free space

Example: Progressive Map Building

As the robot collects more scans, the map becomes more confident and detailed:

Initial (1 scan)

- Mostly unknown (gray)
- Single ray pattern visible
- Low confidence in all cells
- Sparse obstacle detection

Mature (50+ scans)

- Clear obstacle boundaries
- Well-defined free space
- High confidence (sharp black/white)
- Unknown areas only where not explored

Early (10 scans)

- Obstacle outlines emerging
- Free space corridors visible
- Still some uncertainty
- Repeated observations increase confidence

Key Insight

More data = more confidence

Log-odds accumulate evidence over time, making the map increasingly reliable for navigation

Real-World Applications of Occupancy Grids

Warehouse Robots

Amazon, Fetch Robotics: Navigate warehouses, avoid obstacles, plan paths for package delivery

Vacuum Cleaners

Roomba, Roborock: Build floor plans, ensure complete coverage, avoid stairs and obstacles

Autonomous Vehicles

Tesla, Waymo: Local obstacle detection, parking assistance, low-speed maneuvering

Drones and UAVs

Indoor navigation, 3D mapping, inspection tasks in GPS-denied environments

Agriculture

Autonomous tractors and harvesters navigate fields, avoid crops and obstacles

Search and Rescue

Robots explore disaster zones, build maps for first responders, locate survivors

Limitations and Challenges

Dynamic Environments

Moving objects (people, vehicles) cause false obstacles.
Need temporal filtering or object tracking.

Loop Closure Drift

Pose errors accumulate over time. When robot returns to start, map may not align (requires SLAM).

Transparent/Reflective Surfaces

Glass, mirrors, water cause false readings. LiDAR beams pass through or reflect incorrectly.

2D vs 3D Limitation

2D grids miss overhangs, low obstacles. 3D voxel grids solve this but require much more memory.

Computational Cost

Large grids (1000×1000) with high update rates require significant processing power.

Resolution Trade-off

Fine resolution = more detail but more memory/computation. Coarse resolution = less accurate.

Beyond Occupancy Grids

Advanced mapping representations for specific applications:

3D Voxel Grids & Octrees

3D extension of occupancy grids. Octrees provide hierarchical compression for efficient 3D mapping.

Topological Maps

Graph of locations and connections. Compact representation for large environments, used in navigation.

Feature-Based Maps

Store landmarks (corners, edges) instead of grids. Used in visual SLAM, more memory-efficient.

Hybrid Approaches

Combine multiple representations (e.g., local occupancy grid + global topological map) for best of both worlds.

Semantic Maps

Label objects (chair, table, door) using deep learning. Enables high-level reasoning and task planning.

Neural Implicit Maps

Emerging: Use neural networks to represent maps

This Week's Lab: Build Your Own Occupancy Grid

Activity 1: Visualize LiDAR Data

Load simulated LiDAR scans, convert to Cartesian, visualize as point cloud

Activity 2: Basic Occupancy Grid

Implement grid data structure, mark occupied cells, visualize as grayscale image

Activity 3: Full Mapping with Ray Casting

Implement Bresenham ray casting, update with log-odds, build complete map

Deliverables:

- Python code (3 activities)
- Visualizations (plots, maps)
- Analysis answers

Details:

Time: 2-3 hours

Tools: Python, NumPy, Matplotlib

Data: Provided LiDAR scans

Key Takeaways

- ✓ **LiDAR** is the industry standard for robot mapping: accurate, 360° coverage, works in all lighting
- ✓ **Occupancy grids** discretize the environment into cells storing probability of occupancy
- ✓ **Log-odds representation** enables efficient updates using addition instead of multiplication
- ✓ **Inverse sensor model** updates multiple cells per measurement (occupied endpoint + free space along ray)
- ✓ **Ray casting (Bresenham)** identifies cells along a ray for marking free space
- ✓ **Known poses assumption:** This week assumes robot pose is known from localization (Week 3)
- ✓ **Limitations:** Dynamic environments, transparent surfaces, 2D vs 3D, computational cost

Questions?

Let's discuss distance sensing and occupancy grid mapping

Next Week: Path Planning and Navigation

Now that we can localize and map, how do we plan paths to navigate from A to B?